

PostgreSQL

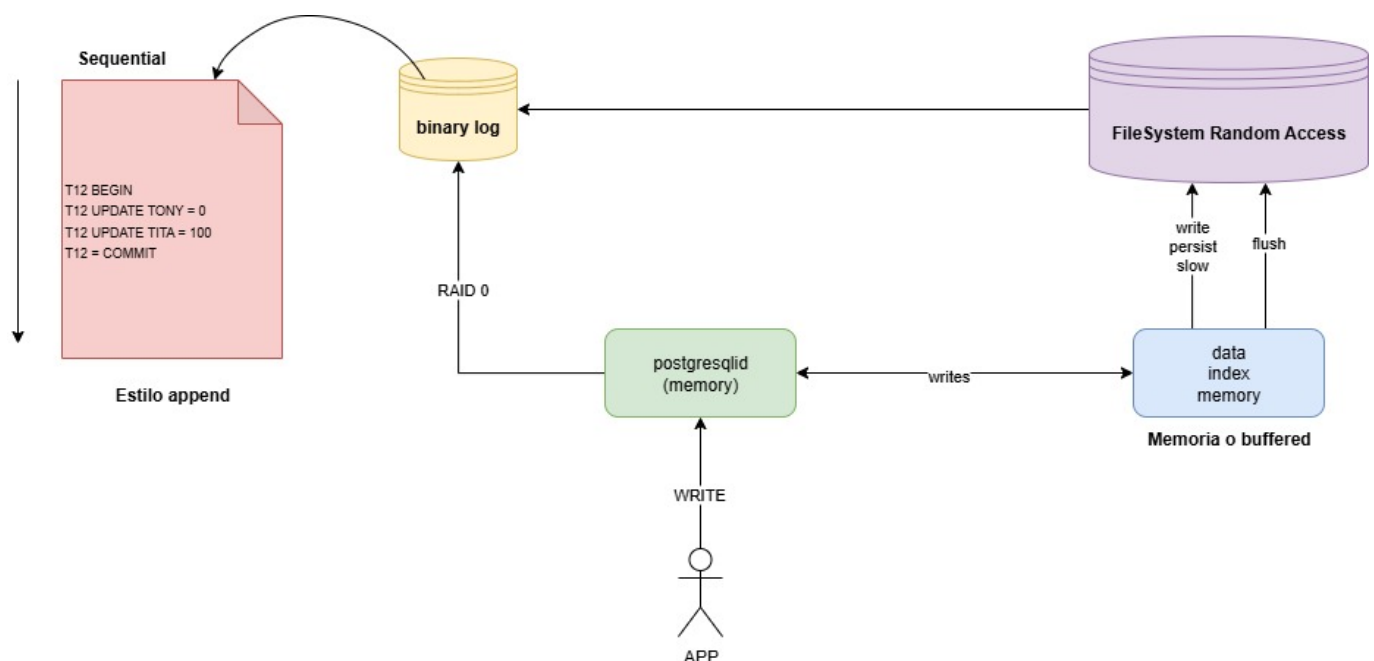
Memoria volatil

- Si la maquina se apaga, se daña etc. Van a perderse los dato almacenados en la memoria. La memoria de dato o de indice se organizan dentro de un file system. Los mismos tendra archivos de random acces. Lo que quiere decir es que las operaciones de escrituras a disco van a persistir y ademas van a ser muy lentas.
- Si quiere cumplir con ACID, una base de datos que esta recibiendo de uina aplicacion muchas escrituras, al ser escritas tienen que ser actualizadas en memoria y ademas en el file system. Si la tabla esta normalizada, se tendran que hacer muchas operaciones a disco que generan que se vuelva lento
- Si te tienen binary log y se quiere cumplir con ACID, el tiempo de escritura va a aumentar mucho por que se tiene que asegurar que los datos fueron escritos a disco principal.
- El binary log da una alternativa y una buena practica, es tener separado los discos a dopnde se pueda posible. Es decir, tener un disco para binary log y otro para los datos. Aunque en el mayoría de setups se encuentran en el mismo disco.

Almacenamiento por binary log

Binary log: Archivo de acceso secuencial. Alamacena por medio de los ID asignados por las bases de datos a las transacciones. Es una herramienta que permite incrementar la velocidad de escritura y lectura sobre la base de datos in tener el riesgo de mucho timeout y garantizando la persistencia de los datos.

- Empiezan con begin, luego todos los statemenos que involucran la transaccion y por ultimo un commit.
- Al ser un archivo secuencial representa que van a ir haciendo appends, es decir anadir al final. Y al ser secuencial se tienen con seguridad de el primer statement fue generado primero que el consecuente
- **Transaccion** -> Es una operacion de modificacion de datos. (No una secuencia de statements)



Los que hace diferente de el almacenamiento del binary log vs el file sistem, el bianry log es un solo archivo sy se va haciendo appends. En el file system, se tiene archivos por tablas etc por ello el almacenamiento e

insercion se hacen siguiendo las reglas de una base de datos tradicional. En el binary simplemente se guardan.

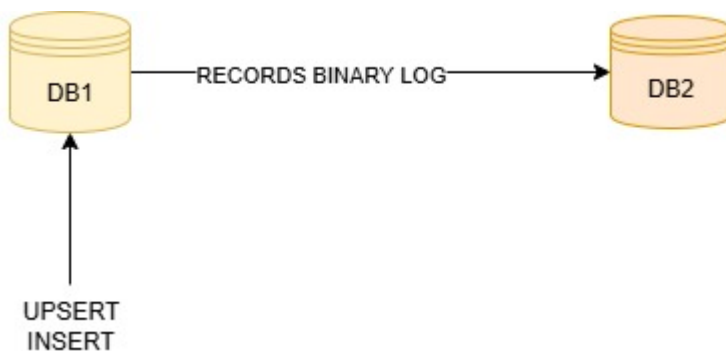
Cuando se agrega la transaccion al binary log (PASO 1) como segundo paso se van a actualizar los datos en memoria principal (Simplemente los datos van a quedar en memoria principal para quedar en el buffer) van estar guardados, teniendo en cuenta que pueden ser volatiles, los va a mantener hasta que se genere el paso 3 FLUSH de los datos o un FSINC, agarran los datos para persistirlos en disco. Y todas las transacciones de memoria van a poner commit en el binary log. Asi es como los datos van a quedar persistentes.

Es relevante pues el tiempo de los datos en buffered puede ser bastante largos y durante este tiempo se recolecta los datos se pudo caer el buffer. Luego la base de datos al reiniciar va a revisar el binary log si hay transacciones que no se les hizo commit o el append de commit, querra decir que los datos no llegaron a disco.

La base de datos va a leer los record sin commit y los va ejecutando en orden para meterlos a disco.

Binary log como mecanismo de replicacion de datos

Entre dos bases de datos una va a estar activa, y la misma va estar enviando los datos a la que esta en stand by. Cuando alguien viene y hace escritura la bd activa pasa los records por medio del binary log. Me garantiza que si las dos bases de datos aplican las mismas operaciones del binary log, en algun momento se van a tener dos bases de datos iguales, teniendo backup y manera de replicar datos.



Collocation

Es un datacenter que se puede alquilar (tiempo antes de la nube). El mismo podía analizar n bases de datos en un motor de búsqueda por ejemplo mysql. Una era la primary y la segunda era la slave/secondary/standby cada base tenía n disco para log, uno para data y otro para binary log. La primary recibía request que se pasaban via streaming hacia el otro servidor. Y se iba a escribir al binary log de la secondary.

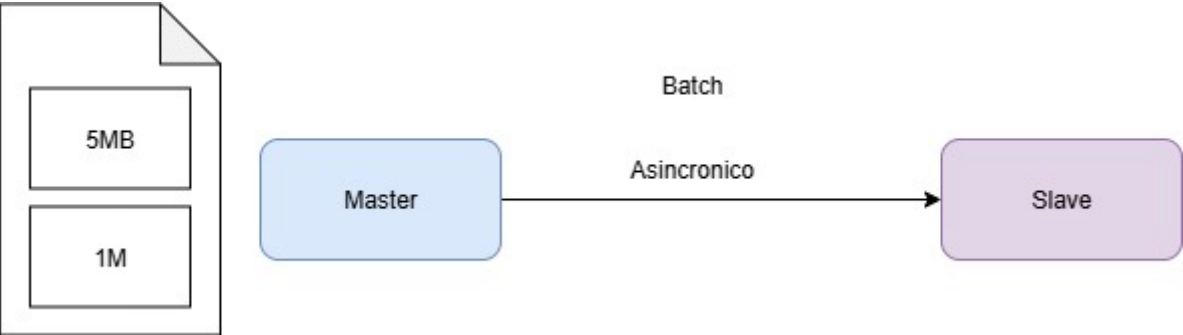
Cuando se tienen un servidor en stand by esperando que el primary falle, se tiene que tener el mismo poder en todas las bases de datos. Además, en el servidor de la base de datos no se puede tener otras apps corriendo (pueden competir con los recursos y puede afectar recursos compartidos). Una solución para evitar este tipo de errores es tener las bases en servidores solas, en caso de un error en binary log ayuda a recuperar la base de datos en caso de desastre.

Master-Slave Replication

Entre las bases de datos se pueden replicar datos. Tiene dos modos de operación:

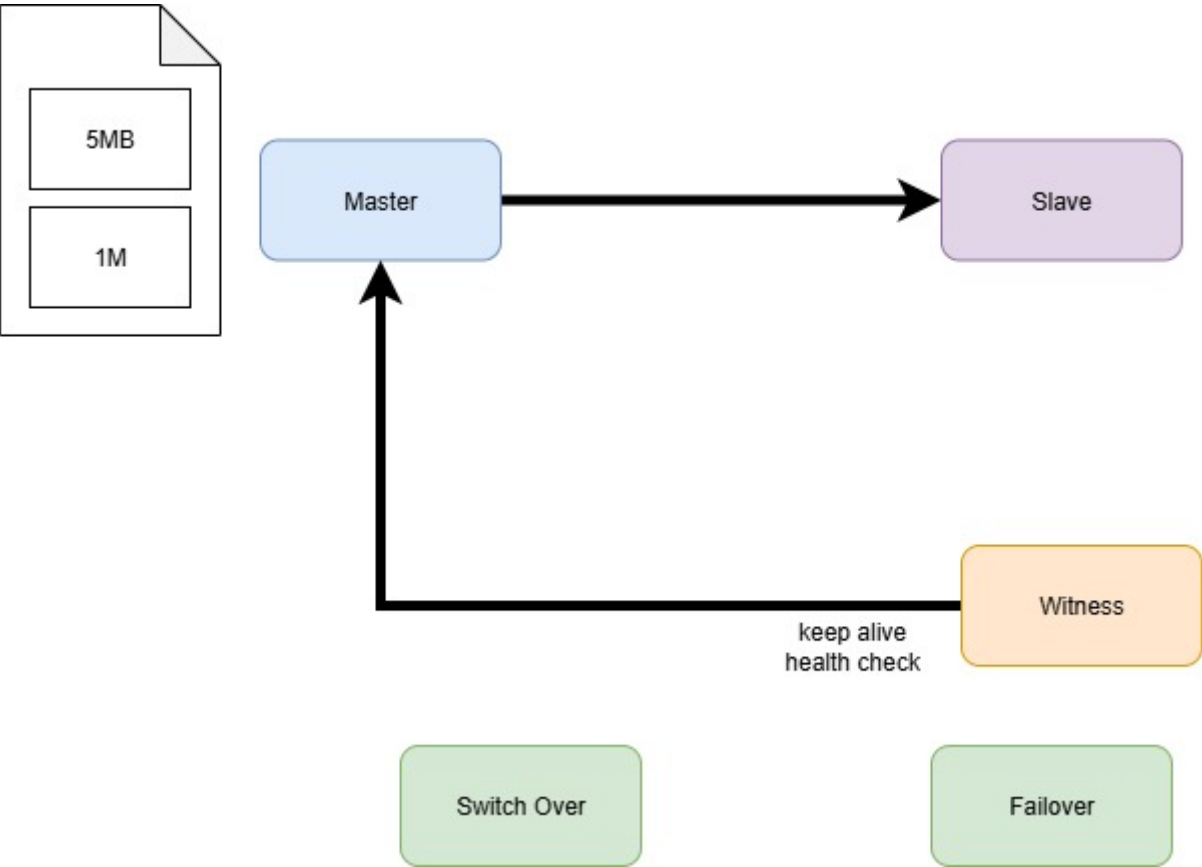
Asincrono

Se habla de tener un envio en batch. Va a completar una ventana de eventos dentro del binary log, es decir cuando se recolecto 5 megas de eventos se hara el envio de manera asincrona. El binary log del slave estara atrasado 5 megas del binary log del master. Se relaciona al RTO y RPO. Pues los mismos como regla de negocio. Esto hace un solo uso de red al tener un envio de golpe. Puede ser riesgoso que la ventana va a poder perder muchos datos si pasa algo.



Sincrono

Apenas llega un evento al binary log, de forma inmediata se envia al binary log. Se gastan mas recursos, pues utiliza la red cada que llega una solicitud.



Switch Over

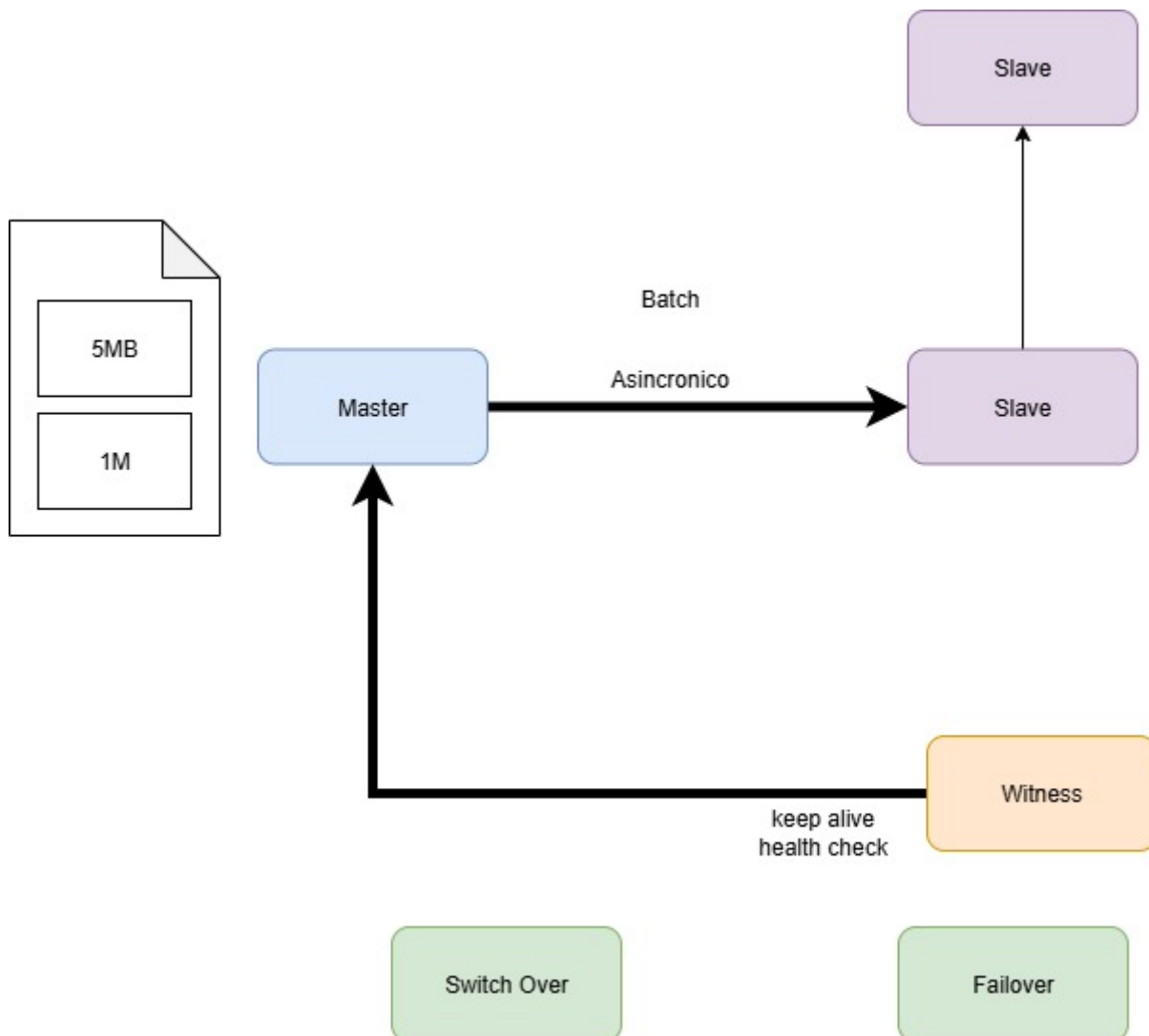
Va permitir cabiar el trafico en caso de que el master falle y lo redirige a otro punto.

Fail Over

Siempre va a necesitar un witness/testigo/arbitro puede ser un slave, v a desempatar el voto pues el failover es automatico, se debe tomar una decision. Tienen una pequena instancia de pocos recursos para el witness, el cual corre pruebas a las bases de datos conocidas como keep alive/health checks permiten determinar si un servidor se cayo. Gracias al witness se podra saber si hay problemas de comunicacion y de donde provienen. Se tendra un load balancer, gateway o proxy haciendo pruebas si las bases estan arriba

Cascada de servidores

El master envia datos al slave, luego el slave lo envia a otros y asi consecuentemente. Aumentando la disponibilidad del servicio, en terminos que se puede llegar y replicar hacia fuera es decir enviar datos a otros nodos.



Data Partitioning -> server owns partition

Se hace particionamiento de datos sobre servidores para sacar el maximos progvecho de hardware disponible conm partiiones funcionando como master y slave. Se utiliza una forma de agarrar una tabla y dividirla en pedacitos basados por una caracteristica que refleje los registros a dividir. Guardando cada division en servidores distintos. Si se quieren hacer replicacion se generan particiones secundarios almacenadas en servidores distintos a la primaria. Los slave no estaran en stand by podran estar listos para consultas. Esta solucion consume reursos en forma overhead.

Overhead

Puede generar el overhead, se hace una tarea pero se gasta tiempo extra en hacer una tarea. Son tiempos y recursos que se gastan pero no se usan directamente en la función primaria de la base de datos.

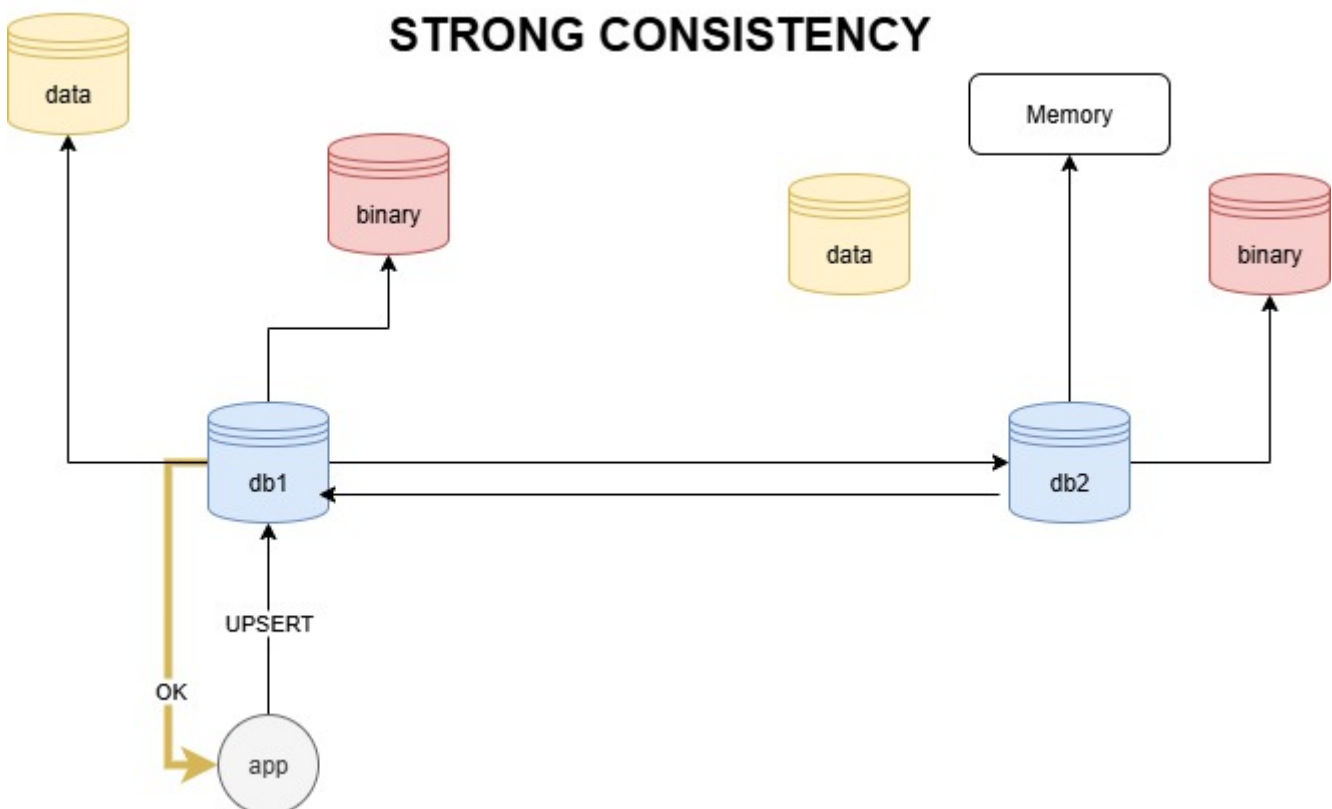
Read Only Replica

Es una base de datos que no recibe updates pero sí recibe consultas. Una consulta de escritura puede enviar actualizaciones a la master y ella a la replica. Se tendrá **consistencia eventual**, si el batch dice que se va a esperar para enviarlos y si alguien mientras tanto hace una consulta en la slave no se recupera los últimos registros.

- No es aplicable para SQL pues no se puede permitir que haya inconsistencia en una base SQL.
- El envío sincrónico de los datos pero no se maneja una confirmación de escritura aun habrían consistencia eventual.

Si quiero mantener consistencia fuerte se debe tener un disco para la data y otro para el binary log. Hay que tener en cuenta que los clientes se quedan esperando una respuesta. Cuando está en modo de binary log, al escribir en él ya se puede saber que los datos son persistentes e indicarle al cliente que se generó bien. Y cuando escribe a la memoria le indica al cliente que todo fue exitoso. Esto sucede en el paso 2 pues se tiene seguridad que el dato está en el binary. Además en paralelo los datos se envían a la slave sin esperar que se envíen a memoria. La slave aplica el binary log, pone en memoria principal y notifica a la master que todo está bien. Después de estos pasos, se le dice a la APP que todo fue exitoso.

El **strong consistency** requiere de esos pasos pagando overhead extra para asegurarse que los datos llegaron bien.



Write Concern -> Cuando se juega con la recepción del cambio, con el tiempo de la persistencia y a cuántas replicas les llega los datos además de cuando los datos me quedan en memoria. Se puede ser completamente fuerte o muy relajado.

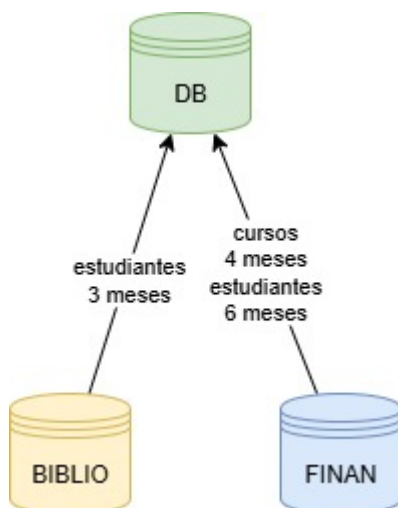
Warm Standby Using Point-In-Time Recovery

Se tienen dos servidores donde se envia info de un servidor al otro, al tener las apps en el master y se quiere correr otra app sobre la misma base de datos y la misma consume muchos recursos, como consecuencia ofusca la base de datos. Como solucion, corta de forma temporal la replica y la convierte en primaria para correr un app sobre la base de datos, no pasa nada pues es una base de datos separada sin vinculos, quedando en un punto del tiempo cuando se desvinculo. Cuando se restablece el vinvulo se envia un segmento de golpe con toda la info del bianry log y se le envia a la replica para que se pueda recuperar.



Logical Replication (only changes)

Se hace repicacion con el mecanismo **publisher/suscriber**, se tiene una base de datos principal conocida como publisher y las otras bases de datos se convierten en un suscriber. Cada base de datos secundaria se conecta a las tablas que necesitan nada mas y recibe actualizaciones cada vez que necesiten. Cada base de datos pueden realizar varias surscripciones con diferentes periodos de replicacion por tiempo.

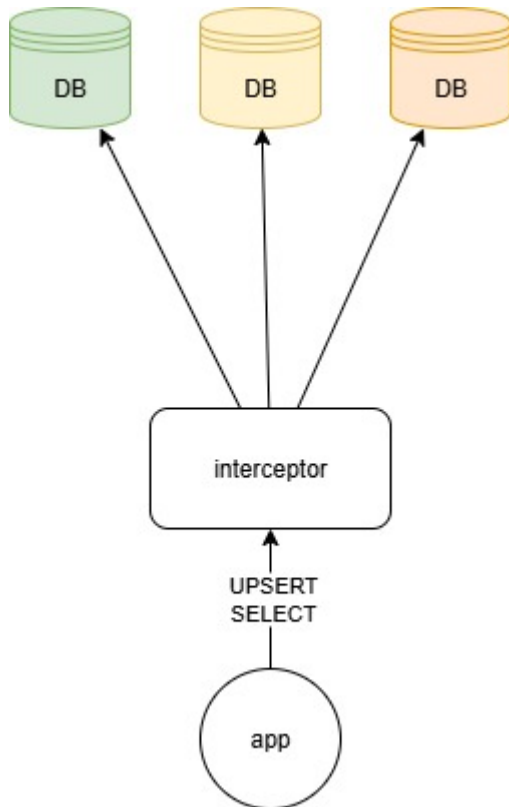


Trigger Based Primary-Standby Replication

Se utiliza una base de datos primary y otras secundarias o en standby que se suscriben a una tabla. Jala la informacion por medio de un trigger es decir una accion por tiempo, este puede ser un cronjob. Si es asincrono espera a enviar la informacion hasta que el trigger se activa.

SQL-BASED Replication Middleware

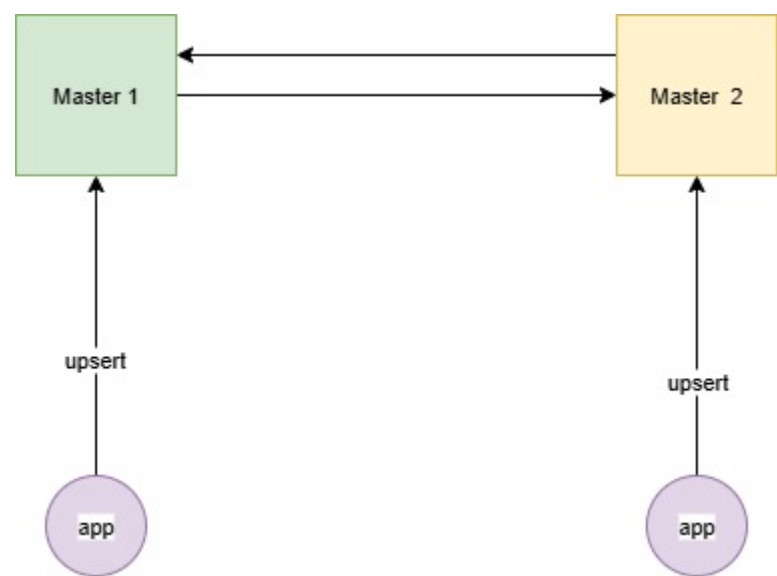
Se tiene servidores independientes, sin saber que los otros existen. Todos son identicos, con componentes separados. En medio tienen un interceptor que es una pieza de software que esta al frente de los servidores, e implementa la misma interfaz que las bases de datos. El interceptor habla el dialecto de la base de datos. Cuando se envia una operacion, las operaciones de modificacion (UPDATE O INSERT) van a enviarse a todos los servidores. Si es un SELECT se balancea la carga y se envia a un solo servidor. con eso se tiene la replicacion de las bases con misma informacion y fuerte consistencia.



Se tienen n servidores master con la misma informacion, se puede tener una apps haciendo insert en las distintas master de manera simultanea, cuando esto sucede se tiene dos modos de operacion.

Asynchronous Multimaster Replication

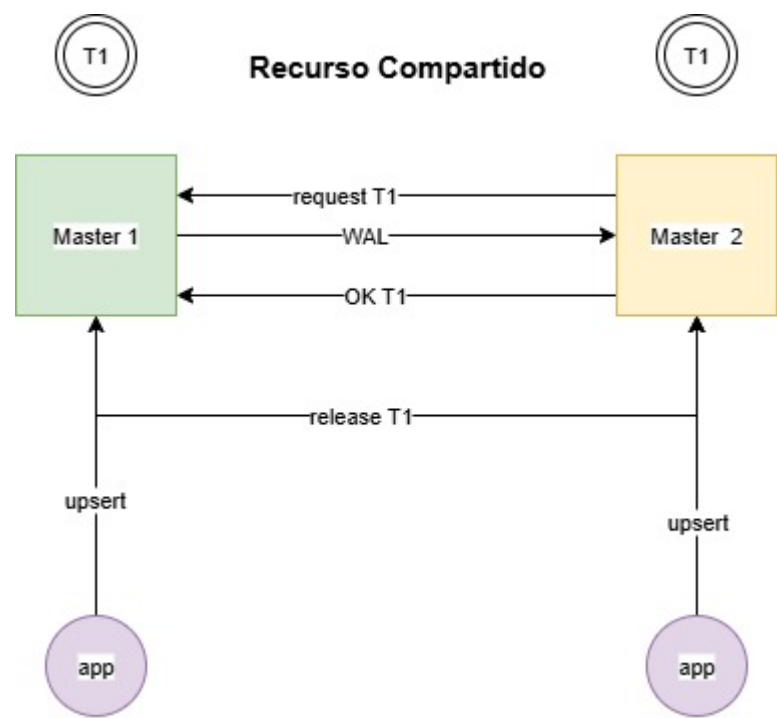
Cada servidor recibe sus inserts y guardan los cambios y cada cierto periodo se sincronizan, en caso de haber un cambio en la misma tabla y row pueden generar conflictos. Pues no es interesante que esto suceda y las bases de datos tienen algoritmos de merge automaticos o se pueden tener intervencion manual.



Synchronous Multimaster Replication

Cada servidor recibe sus inserts y updates; en caso de hacer una misma operacion, las tablas en las dos bases de datos se convierten un shared resource. Para evitar que no haya conflictos e inconsistencia se juega con el tipo de algoritmo. Cuando llega la transaccion el servidor manda request al otro preguntando acerca de los recursos compartidos a utiliza. Se envia y aplica los cambios al otro servidor, y envia un release de T1. Y así con las transacciones al mismo tiempo en la misma unidad de datos. Es decir bloquea los recursos. Como consecuencia sufren los clientes porque tienen que esperar las etapas de sincronización.

Se puede utilizar un esquema **low writes** y **high reads** pues no se genera una sincronización muy grande va a ser poco común algún tiempo de espera.



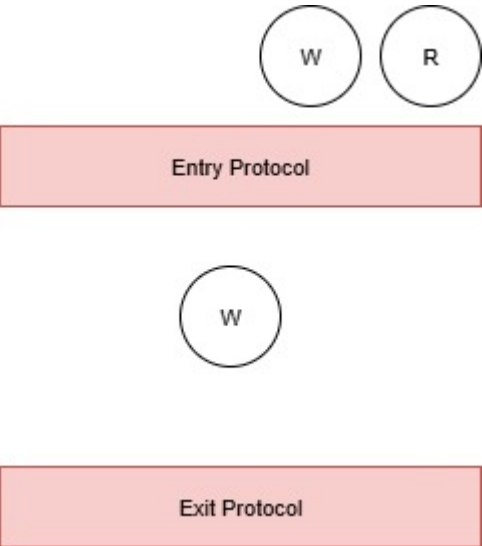
Esquema de operaciones en una base de datos / Region critica

Dentro del esquema se tiene write y read. Las bases de datos SQL manejan exclusividad, es decir una operación de escritura es exclusiva. Quiere decir que si hay una 2 operaciones de escritura sobre el mismo

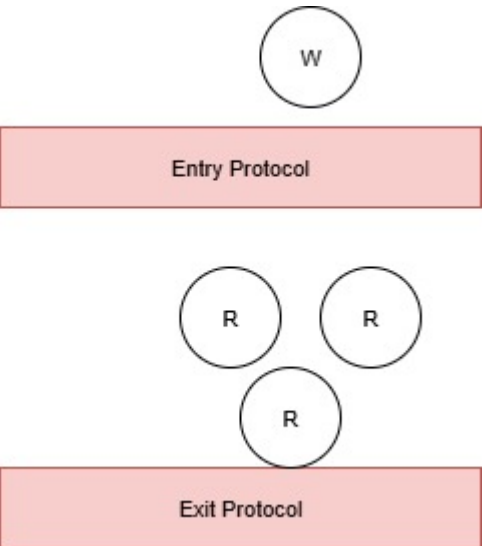
recurso compartido no se podra escribir o leer sobre la operacion durante se escriba, Si se tiene una operacion de lectura, nmo se podria escribir mientras se lle se bloque para writes. Si entra otra lenctura al mismo tiempo, se permite que se de las dos de forma simultanea

	write	read
write	-	-
read	-	+

Se tienen una porion del codigop que tendra un protocolo de entrada y uno de salida. Cuando vienen varias operaciones. Si una operacion write entra a la region critica el procolo de entrada bloquea la entrada y si se dejan entrar otras operaciones genera resultados inconsistentes. Cuando la write sale el porcotolo de salida abre la entrada, y en orden de llegada de las transacciones de espera y se les permite entrar y aplicar el protocolo segun el tipo de operacion.



Si es lectura varias pueden entrar a la operacion critica, pueden generar problemas si se tiene una alto volumen de lecturas la operacion de escritura va quedar esperando que terminen pero a la app que solicito va a dar un timeout.



Para ello se puede tener niveles de accesos en relacion a los casos de usos.

